

Parallelizing Short-time Fourier Transforms

Minh Chau Nguyen

10. Dezember 2024

Introduction

Fourier Transforms

Definition

an integral transform that takes a function as input and outputs another function that describes the extent to which various frequencies are present in the original function

Computation

discrete Fourier transform and fast Fourier transform

Short-time Fourier Transforms

Definition

divide a longer time signal into shorter segments of equal length and then compute the Fourier transform separately on each shorter segment

Computation

- standard methods using FFT algorithm
- iterative methods
- feedforward or pruned methods

Base Code and Profiling

Base Code

```
1 rarray<std::complex<double>, 2> fft::stft_dft(  
2     rarray<std::complex<double>, 1> &vec, size_t window_size, size_t  
3     ↪ window_step)  
4 {  
5     size_t spectrogram_size = floor((vec.size() - window_size) /  
6     ↪ float(window_step)) + 1;  
7     rarray<std::complex<double>, 2> spectrogram(spectrogram_size,  
8     ↪ window_size);  
9  
10    for (size_t i = 0; i < spectrogram_size; i++)  
11    {  
12        rarray<std::complex<double>, 1> dft = fft::dft(vec, i *  
13        ↪ window_step, window_size);  
14  
15        for (size_t j = 0; j < window_size; j++)  
16            spectrogram[i][j] = dft[j];  
17    }  
18    return spectrogram;  
19 }
```

Base Code: DFT

```
1  rarray<std::complex<double>, 1> fft::dft(  
2      const rarray<std::complex<double>, 1> &signal,  
3      size_t vec_begin, size_t vec_size)  
4  {  
5      rarray<std::complex<double>, 1> output(vec_size);  
6      for (size_t k = 0; k < vec_size; k++)  
7      {  
8          std::complex<double> sum = 0.;  
9          for (size_t t = 0; t < vec_size; t++)  
10         {  
11             double angle = 2 * PI * t * k / vec_size;  
12             sum += signal[t + vec_begin] * std::exp(std::complex<double>(0,  
13                 ↪ -angle));  
14         }  
15         output[k] = sum;  
16     }  
17     return output;  
18 }
```

Profiling

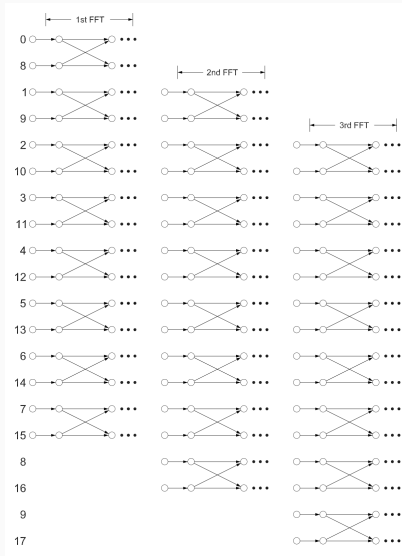
```
1  %   cumulative    self          self     total
2  time   seconds    seconds   calls  us/call  us/call  name
3  100.07    0.35      0.35      745   470.15   470.15
   ↳  fft::dft(ra::rarray<std::complex<double>, 1> const&, unsigned
   ↳  long, unsigned long)
4  0.00      0.35      0.00      747    0.00    0.00
   ↳  ra::detail::Offsets::Offsets(std::vector<long,
   ↳  std::allocator<long> > const&)
```


Implementations

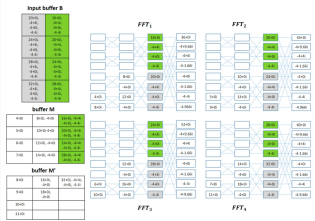
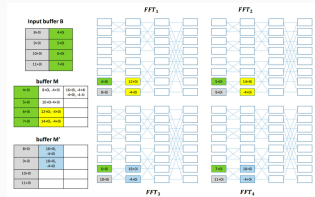
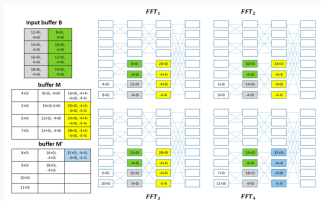
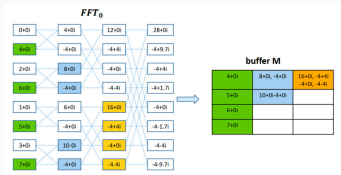
1. Standard method with DFT
2. Standard method with FFT
3. Serial feedforward method
4. Quasi-parallel feedforward method

Feedforward Algorithm

The Idea



The Idea



Profiling

```
1 Each sample counts as 0.01 seconds.
2   % cumulative   self           self   total
3   time  seconds  seconds    calls  ms/call  ms/call  name
4 56.83    102.80    102.80    39024     2.63     2.63
   ↳ ra::rarray<std::complex<double>, 2>::rarray<2, void>(long, long)
5 22.02    142.63     39.83    21463     1.86     1.86
   ↳ ra::rarray<std::complex<double>, 3>::rarray<3, void>(long, long,
   ↳ long)
6 17.04    173.45     30.83
   ↳ fft::stft_qpff(ra::rarray<std::complex<double>, 1>&, unsigned
   ↳ long, unsigned long)
7  4.08    180.84      7.38    19510     0.38     3.01
   ↳ fft::stft_qpff_batch(ra::rarray<std::complex<double>, 3>,
   ↳ ra::rarray<std::complex<double>, 2>, unsigned long, unsigned
   ↳ long)
8  0.04    180.92      0.08
   ↳ ra::detail::Offsets::Offsets(std::vector<long,
   ↳ std::allocator<long> > const&)
9  0.00    180.92      0.00        1     0.00     0.00
   ↳ _GLOBAL__sub_I__ZN3fft8stft_dftERN2ra6rarrayIst7complexIdELi1EEEmm
10 0.00    180.92      0.00        1     0.00     0.00
   ↳ fft::transform(ra::rarray<std::complex<double>, 1>&,
   ↳ ra::rarray<std::complex<double>, 2>&,
   ↳ ra::rarray<std::complex<double>, 3>&)
```

Shared-memory Parallelism

Standard method with DFT: Code

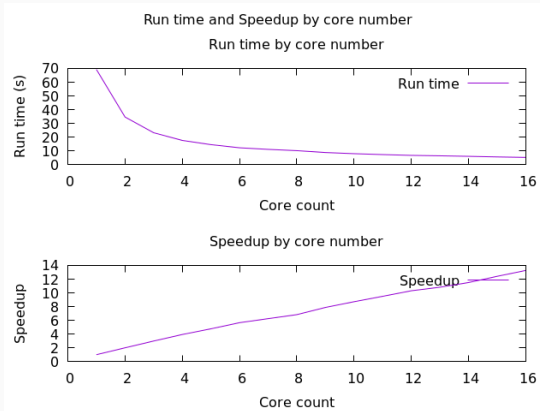
```
1  rarray<std::complex<double>, 2> fft::stft_dft(  
2      rarray<std::complex<double>, 1> &vec, size_t window_size, size_t  
    ↪ window_step)  
3  {  
4      ...  
5      #pragma omp parallel for default(none) shared(vec, window_size,  
    ↪ window_step, spectrogram, spectrogram_size)  
6      for (size_t i = 0; i < spectrogram_size; i++)  
7      {  
8          rarray<std::complex<double>, 1> dft = fft::dft(vec, i *  
    ↪ window_step, window_size);  
9  
10     #pragma omp parallel for default(none) shared(window_size, dft, i,  
    ↪ spectrogram)  
11     for (size_t j = 0; j < window_size; j++)  
12         spectrogram[i][j] = dft[j];  
13     }  
14     return spectrogram;  
15 }
```

Standard method with DFT: Code

```
1 rarray<std::complex<double>, 1> fft::dft(
2     const rarray<std::complex<double>, 1> &signal,
3     size_t vec_begin, size_t vec_size)
4 {
5     rarray<std::complex<double>, 1> output(vec_size);
6     #pragma omp parallel for default(none) shared(signal, vec_begin,
7         ↪ vec_size, output)
8     for (size_t k = 0; k < vec_size; k++)
9     {
10         std::complex<double> sum = 0.;
11         for (size_t t = 0; t < vec_size; t++)
12         {
13             double angle = 2 * PI * t * k / vec_size;
14             sum += signal[t + vec_begin] * std::exp(std::complex<double>(0,
15                 ↪ -angle));
16         }
17         output[k] = sum;
18     }
19     return output;
20 }
```


Standard method with DFT: Scaling analysis

Run scaling analysis job for 1 to 16 cores with `num-samples=10000` and `window-size=256`.



Standard method with FFT: Code

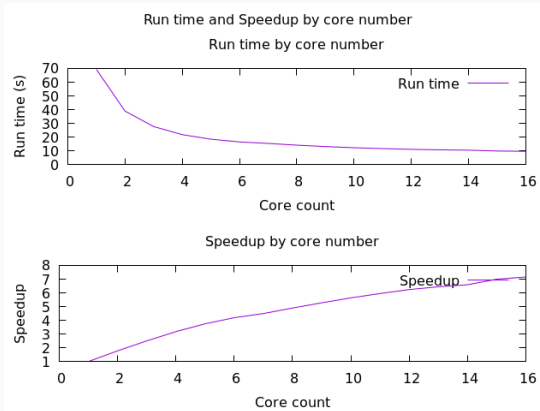
```
1  rarray<std::complex<double>, 2> fft::stft_fft(  
2      rarray<std::complex<double>, 1> &vec, size_t window_size, size_t  
    ⇨ window_step)  
3  {  
4      ...  
5      #pragma omp parallel for default(none) shared(vec, window_size,  
    ⇨ window_step, spectrogram, spectrogram_size)  
6      for (size_t i = 0; i < spectrogram_size; i++)  
7      {  
8          rarray<std::complex<double>, 1> fft = fft::fft(vec, i *  
    ⇨ window_step, window_size);  
9  
10     #pragma omp parallel for default(none) shared(window_size, fft, i,  
    ⇨ spectrogram)  
11     for (size_t j = 0; j < window_size; j++)  
12         spectrogram[i][j] = fft[j];  
13     }  
14     return spectrogram;  
15 }
```

Standard method with FFT: Code

```
1 rarray<std::complex<double>, 1> fft::fft(  
2     const rarray<std::complex<double>, 1> &signal,  
3     size_t vec_begin, size_t vec_size)  
4 {  
5     rarray<std::complex<double>, 1> subsignal(vec_size);  
6     #pragma omp parallel for default(none) shared(signal, vec_begin,  
7         ↪ vec_size, subsignal)  
8     for (size_t i = 0; i < vec_size; i++)  
9         subsignal[i] = signal[vec_begin + i];  
10  
11     fft::transform(subsignal);  
12     return subsignal;  
13 }
```

Standard method with FFT: Scaling analysis

Run scaling analysis job for 1 to 16 cores with
`num-samples=1000000` and `window-size=1024`.

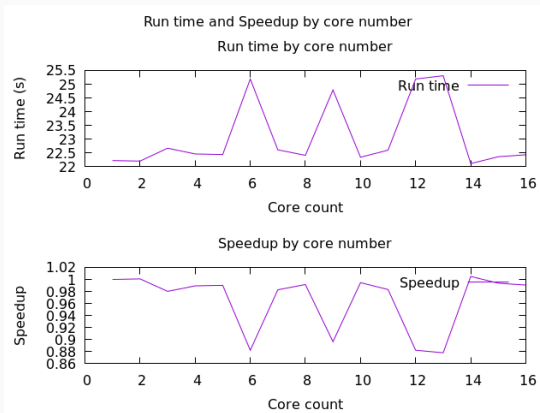


Serial feedforward method: Code

```
1  rarray<std::complex<double>, 2> fft::stft_ff(
2      rarray<std::complex<double>, 1> &vec, size_t window_size, size_t
        ↳ window_step)
3  {
4      ...
5      // Perform FFT on 1st N signals
6      rarray<std::complex<double>, 1> subvec(window_size);
7      #pragma omp parallel for default(none) shared(window_size, subvec, vec)
8      for (size_t i = 0; i < window_size; i++)
9          subvec[i] = vec[i];
10     transform(subvec, fft, tw);
11     #pragma omp parallel for default(none) shared(window_size, spectrogram,
        ↳ fft, num_stages)
12     for (size_t i = 0; i < window_size; i++)
13         spectrogram[0][i] = fft[i][num_stages - 1];
14     // Prepare buffer
15     #pragma omp parallel for default(none) shared(num_stages, window_size,
        ↳ buffer, fft)
16     for (size_t stage = 0; stage < num_stages; stage++)
17         ...
18     // Serial part...
19     return spectrogram;
20 }
```

Serial feedforward method: Scaling analysis

Run scaling analysis job for 1 to 16 cores with
`num-samples=1000000` and `window-size=1024`.



Serial feedforward method: Code

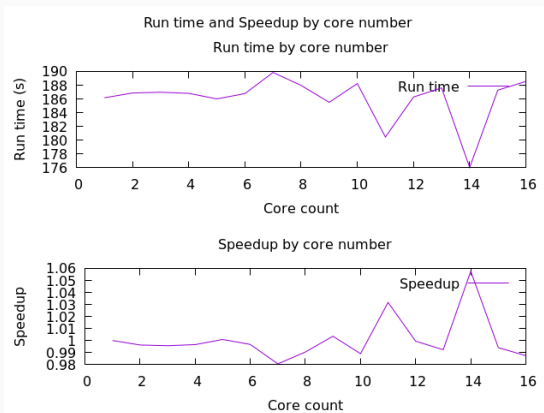
```
1  rarray<std::complex<double>, 2> fft::stft_qpff(
2      rarray<std::complex<double>, 1> &vec, size_t window_size, size_t
        ↪ window_step)
3  {
4      ...
5      // Perform FFT on 1st N signals
6      rarray<std::complex<double>, 1> subvec(window_size);
7      #pragma omp parallel for default(none) shared(window_size, subvec, vec)
8      for (size_t i = 0; i < window_size; i++)
9          subvec[i] = vec[i];
10     transform(subvec, fft, tw);
11     #pragma omp parallel for default(none) shared(window_size, spectrogram,
        ↪ fft, num_stages)
12     for (size_t i = 0; i < window_size; i++)
13         spectrogram[0][i] = fft[i][num_stages - 1];
14     // Initialize M
15     #pragma omp parallel for default(none) shared(num_stages, window_size,
        ↪ M, fft)
16     for (size_t s = 0; s < num_stages - 1; s++)
17         ...
18     // Serial part with a parallel routine
19     return spectrogram;
20 }
```

Quasi-parallel feedforward method: Code

```
1 rarray<std::complex<double>, 2> fft::stft_qpff_batch(
2     rarray<std::complex<double>, 3> B, rarray<std::complex<double>, 2>
3     ↪ tw,
4     size_t window_size, size_t s)
5 {
6     size_t count = (1 << s);
7     rarray<std::complex<double>, 2> f(window_size / 2, window_size);
8
9     #pragma omp parallel default(none) shared(B, tw, window_size, s, count,
10     ↪ f)
11     for (size_t i = 0; i < window_size / 2; i++)
12         for (size_t k = 0; k < count; k++)
13             {
14                 std::complex<double> xr, x0, x1;
15                 xr = B[i][0][k] * tw[window_size - (1 << (s + 1)) + k][s];
16                 x0 = B[i][1][k] + xr;
17                 x1 = B[i][1][k] - xr;
18                 f[i][k] = x0;
19                 f[i][k + count] = x1;
20             }
21     return f;
22 }
```


Quasi-parallel feedforward method: Scaling analysis

Run scaling analysis job for 1 to 16 cores with
`num-samples=1000000` and `window-size=1024`.



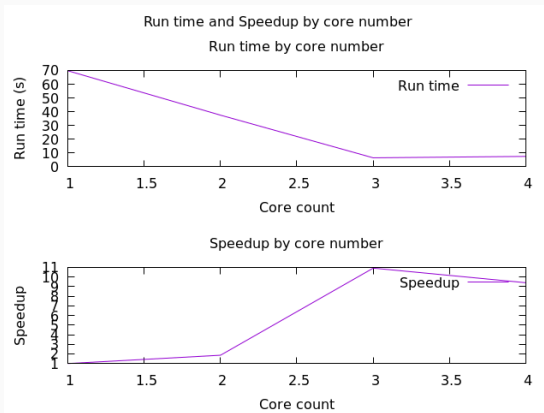
Distributed-memory Parallelism

Standard method with DFT: Code

```
1 rarray<std::complex<double>, 2> fft_mpi::stft_dft(  
2     rarray<std::complex<double>, 1> &vec,  
3     size_t window_size, size_t window_step)  
4 {  
5     ...  
6     int rank, size;  
7     MPI_Init(NULL, NULL);  
8     MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
9     MPI_Comm_size(MPI_COMM_WORLD, &size);  
10    ...  
11    for (size_t i = start_idx; i < end_idx; i++)  
12        ...  
13    if (rank == 0)  
14        for (int p = 1; p < size; p++)  
15            ...  
16            MPI_Recv(&spectrogram[p * chunk_size][0], received_chunk_size *  
17                ↪ window_size, MPI_DOUBLE_COMPLEX, p, 0, MPI_COMM_WORLD,  
18                ↪ MPI_STATUS_IGNORE);  
19    else  
20        MPI_Send(&spectrogram[start_idx][0], (end_idx - start_idx) *  
21            ↪ window_size, MPI_DOUBLE_COMPLEX, 0, 0, MPI_COMM_WORLD);  
22    MPI_Finalize();  
23    return spectrogram;  
24 }
```

Standard method with DFT: Scaling analysis

Run scaling analysis job for 1 to 4 processes with
`num-samples=10000` and `window-size=256`.

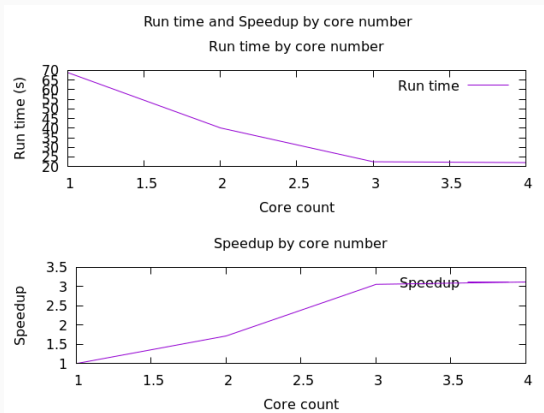


Standard method with FFT: Code

```
1 rarray<std::complex<double>, 2> fft_mpi::stft_fft(
2     rarray<std::complex<double>, 1> &vec, size_t window_size, size_t
   ↪ window_step)
3 {
4     ...
5     int rank, size;
6     MPI_Init(NULL, NULL);
7     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
8     MPI_Comm_size(MPI_COMM_WORLD, &size);
9     ...
10    for (size_t i = start_idx; i < end_idx; i++)
11        ...
12    if (rank == 0)
13        for (int p = 1; p < size; p++)
14            ...
15            MPI_Recv(&spectrogram[p * chunk_size][0], received_chunk_size *
   ↪ window_size, MPI_DOUBLE_COMPLEX, p, 0, MPI_COMM_WORLD,
   ↪ MPI_STATUS_IGNORE);
16    else
17        MPI_Send(&spectrogram[start_idx][0], (end_idx - start_idx) *
   ↪ window_size, MPI_DOUBLE_COMPLEX, 0, 0, MPI_COMM_WORLD);
18    MPI_Finalize();
19    return spectrogram;
}
```

Standard method with DFT: Scaling analysis

Run scaling analysis job for 1 to 4 processes with
`num-samples=1000000` and `window-size=1024`.



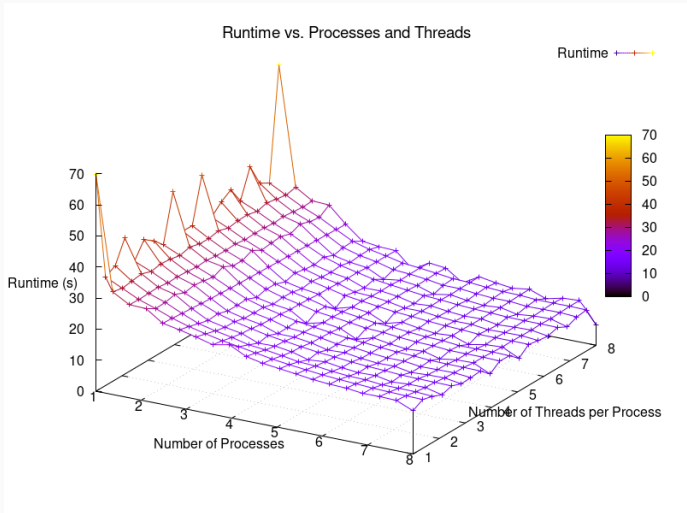
Hybrid Parallelism (OpenMP + MPI)

Standard method with DFT: Code

```
1 rarray<std::complex<double>, 2> fft_mpi::stft_dft(
2     rarray<std::complex<double>, 1> &vec,
3     size_t window_size, size_t window_step)
4 {
5     ...
6     int rank, size;
7     MPI_Init(NULL, NULL);
8     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9     MPI_Comm_size(MPI_COMM_WORLD, &size);
10    ...
11    for (size_t i = start_idx; i < end_idx; i++)
12        ...
13    if (rank == 0)
14        for (int p = 1; p < size; p++)
15            ...
16            MPI_Recv(&spectrogram[p * chunk_size][0], received_chunk_size *
17                ↪ window_size, MPI_DOUBLE_COMPLEX, p, 0, MPI_COMM_WORLD,
18                ↪ MPI_STATUS_IGNORE);
19    else
20        MPI_Send(&spectrogram[start_idx][0], (end_idx - start_idx) *
21            ↪ window_size, MPI_DOUBLE_COMPLEX, 0, 0, MPI_COMM_WORLD);
22    MPI_Finalize();
23    return spectrogram;
24 }
```


Standard method with DFT: Scaling analysis

Run scaling analysis job for 1 to 8 processes, each process with 1 to 8 threads with `num-samples=10000` and `window-size=256`.



Standard method with DFT: Scaling analysis

Run scaling analysis job for 1 to 8 processes, each process with 1 to 8 threads with `num-samples=1000000` and `window-size=1024`.

